

Java Program ក្រុម 7 : Emergency Call Dispatch Queue

EmergencyCallDispatch.java

```
import java.util.*;

public class EmergencyCallDispatch {

    // =====
    // Event Class
    // =====
    static class Event {
        private String eventId;
        private String eventType;
        private String time;
        private int severity;
        private String location;
        private int sequence;

        public Event(String eventId, String eventType, String time,
            int severity, String location, int sequence) {
            this.eventId = eventId;
            this.eventType = eventType;
            this.time = time;
            this.severity = severity;
            this.location = location;
            this.sequence = sequence;
        }

        public String getEventId() {
```

```
        return eventId;
    }

    public String getEventType() {
        return eventType;
    }

    public String getTime() {
        return time;
    }

    public int getSeverity() {
        return severity;
    }

    public String getLocation() {
        return location;
    }

    public int getSequence() {
        return sequence;
    }

    public String getSeverityName() {
        switch (severity) {
            case 1:
                return "Critical";
            case 2:
                return "High";
```

```

        case 3:
            return "Medium";

        case 4:
            return "Low";

        default:
            return "Unknown";

    }

}

@Override
public String toString() {
    return eventId + " | "
        + eventType + " | "
        + time + " | "
        + getSeverityName() + " | "
        + location;

}

}

// =====
// Algorithm A: FIFO Queue
// =====
public static void fifoAlgorithm(List<Event> events) {

    Queue<Event> queue = new ArrayDeque<>();

    System.out.println();
    System.out.println("=====");
    System.out.println("    ALGORITHM A: FIFO QUEUE");

```

```
System.out.println("=====");
```

```
// ENQUEUE
```

```
for (Event event : events) {  
    queue.offer(event);  
}
```

```
System.out.println();
```

```
System.out.println("Queue after ENQUEUE:");  
display(queue);
```

```
// PEEK
```

```
if (!queue.isEmpty()) {  
    System.out.println();  
    System.out.println("PEEK: " + queue.peek());  
}
```

```
// SEARCH
```

```
System.out.println();  
System.out.println("SEARCH E3:");  
searchEvent(queue, "E3");
```

```
// DEQUEUE
```

```
System.out.println();  
System.out.println("Processing Order:");
```

```
while (!queue.isEmpty()) {  
    Event event = queue.poll();  
    System.out.println("DEQUEUE -> " + event);
```

```

    }

    System.out.println();
    System.out.println("Queue after processing: []");
}

// =====
// Algorithm B: Priority Queue
// =====

public static void priorityAlgorithm(List<Event> events) {

    PriorityQueue<Event> queue =
        new PriorityQueue<>(
            Comparator.comparingInt(Event::getSeverity)
                .thenComparingInt(Event::getSequence)
        );

    System.out.println();
    System.out.println("=====");
    System.out.println("  ALGORITHM B: PRIORITY QUEUE");
    System.out.println("=====");

    // ENQUEUE
    for (Event event : events) {
        queue.offer(event);
    }

    System.out.println();
    System.out.println("Queue after ENQUEUE:");
}

```

```
displayPriority(queue);

// PEEK
if (!queue.isEmpty()) {
    System.out.println();
    System.out.println("PEEK: " + queue.peek());
}

// SEARCH
System.out.println();
System.out.println("SEARCH E3:");
searchEvent(queue, "E3");

// DEQUEUE
System.out.println();
System.out.println("Processing Order:");

while (!queue.isEmpty()) {
    Event event = queue.poll();
    System.out.println("DEQUEUE -> " + event);
}

System.out.println();
System.out.println("Queue after processing: []");
}

// =====
// Search Event
// =====
```

```

public static boolean searchEvent(
    Collection<Event> queue, String eventId) {

    for (Event event : queue) {

        if (event.getEventId().equals(eventId)) {
            System.out.println("Event found: " + event);
            return true;
        }
    }

    System.out.println("Event not found: " + eventId);
    return false;
}

// =====
// Cancel Event
// =====

public static boolean cancelEvent(
    Queue<Event> queue, String eventId) {

    Iterator<Event> iterator = queue.iterator();

    while (iterator.hasNext()) {

        Event event = iterator.next();

        if (event.getEventId().equals(eventId)) {
            iterator.remove();

```

```

        System.out.println(
            "Event cancelled successfully: " + eventId);

        return true;
    }
}

System.out.println(
    "Event not found: " + eventId);

return false;
}

// =====
// Display FIFO Queue
// =====
public static void display(Collection<Event> queue) {

    System.out.println("[ ");

    for (Event event : queue) {
        System.out.println(" " + event);
    }

    System.out.println("]");
}

// =====

```

```

// Display Priority Queue

// =====

public static void displayPriority(
    PriorityQueue<Event> queue) {

    PriorityQueue<Event> copy =
        new PriorityQueue<>(queue);

    System.out.println("[ ");

    while (!copy.isEmpty()) {
        System.out.println(" " + copy.poll());
    }

    System.out.println("]");
}

// =====

// Main Method

// =====

public static void main(String[] args) {

    List<Event> events = new ArrayList<>();

    // Event ID, Type, Time, Severity, Location, Sequence

    events.add(new Event(
        "E1", "Medical", "10:01",
        3, "Building A", 1));

```

```
events.add(new Event(
    "E2", "General Incident", "10:02",
    4, "Road B", 2));
```

```
events.add(new Event(
    "E3", "Accident", "10:03",
    1, "Road C", 3));
```

```
events.add(new Event(
    "E4", "Fire", "10:04",
    2, "Building D", 4));
```

```
events.add(new Event(
    "E5", "Accident", "10:05",
    1, "Road E", 5));
```

```
events.add(new Event(
    "E6", "Medical", "10:06",
    3, "Building F", 6));
```

```
// =====
```

```
// Display All Events
```

```
// =====
```

```
System.out.println("=====");
```

```
System.out.println("  EMERGENCY CALL DISPATCH SYSTEM");
```

```
System.out.println("=====");
```

```

System.out.println();

System.out.println("Emergency Events:");


for (Event event : events) {
    System.out.println(event);
}


// =====
// Run Algorithm A
// =====

fifoAlgorithm(events);


// =====
// Run Algorithm B
// =====

priorityAlgorithm(events);


// =====
// Cancel Test
// =====


System.out.println();
System.out.println("=====");
System.out.println("    CANCEL EVENT TEST");
System.out.println("=====");


Queue<Event> cancelQueue = new ArrayDeque<>();

```

```
        for (Event event : events) {
            cancelQueue.offer(event);
        }

        System.out.println();
        System.out.println("Queue before cancellation:");
        display(cancelQueue);

        cancelEvent(cancelQueue, "E3");

        System.out.println();
        System.out.println("Queue after cancelling E3:");
        display(cancelQueue);

        cancelEvent(cancelQueue, "E99");
    }
}
```

ผลลัพธ์ที่ได้

```
=====
EMERGENCY CALL DISPATCH SYSTEM
=====

Emergency Events:
E1 | Medical | 10:01 | Medium | Building A
E2 | General Incident | 10:02 | Low | Road B
E3 | Accident | 10:03 | Critical | Road C
E4 | Fire | 10:04 | High | Building D
E5 | Accident | 10:05 | Critical | Road E
```

E6 | Medical | 10:06 | Medium | Building F

=====

ALGORITHM A: FIFO QUEUE

=====

Queue after ENQUEUE:

[

E1 | Medical | 10:01 | Medium | Building A

E2 | General Incident | 10:02 | Low | Road B

E3 | Accident | 10:03 | Critical | Road C

E4 | Fire | 10:04 | High | Building D

E5 | Accident | 10:05 | Critical | Road E

E6 | Medical | 10:06 | Medium | Building F

]

PEEK: E1 | Medical | 10:01 | Medium | Building A

SEARCH E3:

Event found: E3 | Accident | 10:03 | Critical | Road C

Processing Order:

DEQUEUE -> E1 | Medical | 10:01 | Medium | Building A

DEQUEUE -> E2 | General Incident | 10:02 | Low | Road B

DEQUEUE -> E3 | Accident | 10:03 | Critical | Road C

DEQUEUE -> E4 | Fire | 10:04 | High | Building D

DEQUEUE -> E5 | Accident | 10:05 | Critical | Road E

DEQUEUE -> E6 | Medical | 10:06 | Medium | Building F

Queue after processing: []

=====

ALGORITHM B: PRIORITY QUEUE

=====

Queue after ENQUEUE:

[

E3 | Accident | 10:03 | Critical | Road C

E5 | Accident | 10:05 | Critical | Road E

E4 | Fire | 10:04 | High | Building D

E1 | Medical | 10:01 | Medium | Building A

E6 | Medical | 10:06 | Medium | Building F

E2 | General Incident | 10:02 | Low | Road B

]

PEEK: E3 | Accident | 10:03 | Critical | Road C

SEARCH E3:

Event found: E3 | Accident | 10:03 | Critical | Road C

Processing Order:

DEQUEUE -> E3 | Accident | 10:03 | Critical | Road C

DEQUEUE -> E5 | Accident | 10:05 | Critical | Road E

DEQUEUE -> E4 | Fire | 10:04 | High | Building D

DEQUEUE -> E1 | Medical | 10:01 | Medium | Building A

DEQUEUE -> E6 | Medical | 10:06 | Medium | Building F

DEQUEUE -> E2 | General Incident | 10:02 | Low | Road B

Queue after processing: []

=====

CANCEL EVENT TEST

=====

Queue before cancellation:

[

E1 | Medical | 10:01 | Medium | Building A
E2 | General Incident | 10:02 | Low | Road B
E3 | Accident | 10:03 | Critical | Road C
E4 | Fire | 10:04 | High | Building D
E5 | Accident | 10:05 | Critical | Road E
E6 | Medical | 10:06 | Medium | Building F

]

Event cancelled successfully: E3

Queue after cancelling E3:

[

E1 | Medical | 10:01 | Medium | Building A
E2 | General Incident | 10:02 | Low | Road B
E4 | Fire | 10:04 | High | Building D
E5 | Accident | 10:05 | Critical | Road E
E6 | Medical | 10:06 | Medium | Building F

]

Event not found: E99